

# Mitigating Bias in Technical Recruitment through a Unified, Dynamic Assessment Platform

**Department of Computer Science and Engineering**  
**Graphic Era (Deemed to be University)**  
**Dehradun, Uttarakhand**  
**September-2025**

## CANDIDATE'S DECLARATION

We hereby certify that the work which is being presented in the Synopsis entitled “**Mitigating Bias in Technical Recruitment through a Unified, Dynamic Assessment Platform**” in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering (**with Artificial Intelligence and Machine Learning**) in the Department of Computer Science and Engineering of the Graphic Era (Deemed to be University), Dehradun shall be carried out by the undersigned under the supervision of **Mr. Chitransh Bose, Assistant Professor**, Department of Computer Science and Engineering, Graphic Era (Deemed to be University), Dehradun.

|                          |         |
|--------------------------|---------|
| Akula Sai Santhosh Kumar | 2021791 |
| Ashutosh Munindra Jha    | 2021798 |
| Rahul Negi               | 2021823 |

The above-mentioned students shall be working under the supervision of the undersigned on the “**Mitigating Bias in Technical Recruitment through a Unified, Dynamic Assessment Platform**”

Signature  
Supervisor

Signature  
Head of the Department

### Internal Evaluation (By DPRC Committee)

**Status of the Synopsis:** Accepted / Rejected

**Any Comments:**

**Name of the Committee Members:**

**Signature with Date**

- 1.
- 2.

# Table of Contents

---

| Chapter No. | Description                        | Page No. |
|-------------|------------------------------------|----------|
| Chapter 1   | Introduction and Problem Statement | 1        |
| Chapter 2   | Background/ Literature Survey      | 3        |
| Chapter 3   | Objectives                         | 6        |
| Chapter 4   | Hardware and Software Requirements | 7        |
| Chapter 5   | Possible Approach/ Algorithms      | 8        |
|             | References                         | 11       |

# Chapter 1

## Introduction and Problem Statement

### 1.1 Introduction

Hiring for tech roles is a massive headache because recruiters get buried under hundreds of applications for just one position, making manual screening a slow and subjective chore. The old ways of hiring struggle to show the whole picture of a candidate, often judging technical skills and personality traits separately. Today's automated tools aren't much better, as they usually focus on just one thing, like matching keywords on a resume or hosting a coding test. This scattered approach makes for a clumsy process for applicants and gives recruiters an incomplete view, making it hard to efficiently find the best person for the job. We really need a single, all-in-one platform that automates the screening process to provide a full, data-backed look at every single applicant.

This is where our project, the Mitigating Bias in Technical Recruitment through a Unified, Dynamic Assessment Platform, comes in. It's an all-in-one AI system designed to smooth out the early steps of hiring by combining three key assessments into one simple workflow. First, it reads and understands a candidate's resume. The clever part is that it then uses that info to create personalized interview questions specifically for that applicant, making the screening more insightful. After that, it gives them an automated coding challenge to objectively test their technical skills. By putting all these pieces together, the platform gives recruiters a clear, simple dashboard with a single compatibility score, helping them make hiring decisions that are faster, smarter and far less biased.

### 1.2 Problem Statement

The main problem with tech recruiting is the lack of a smart, all-in-one system to properly size up candidates. This forces recruiters to waste countless hours manually screening resumes, leading to biased decisions and a broken hiring process that fails to measure both a person's coding abilities and their soft skills together.

This broken process creates three major problems:

**1.2.1 Wasted Time:** Recruiters and senior engineers get bogged down in repetitive screening work instead of talking with the best candidates.

**1.2.2 Bias and Unfairness:** Too often, hiring decisions are based on a "gut feeling" rather than on real data, which lets unconscious biases control the outcome.

**1.2.3 An Incomplete Puzzle:** Recruiters are left trying to patch together a candidate's profile from different sources, making it easy to misjudge them or miss their full potential.

Our project will fix this by building a single platform that automatically analyzes a resume, creates unique interview questions based on that resume and includes coding challenges. The goal is to deliver a complete, fair, and unbiased look to every candidate.

## **Chapter 2**

### **Background/ Literature Survey**

In this chapter, we review existing work in the key areas related to this project: automated resume parsing, AI-powered video interviews and online coding assessment platforms.

#### **2.1 Automated Resume Parsing**

Teaching computers to automatically pull information from resumes is a major focus in the field of Natural Language Processing (NLP). Older systems weren't very flexible; they relied on simple keyword searching or rigid templates and would break if a resume's format was unusual. "The modern way is much smarter. It uses machine learning techniques like Named Entity Recognition (NER) to intelligently identify and classify key information like skills, company names and job titles. The approach detailed by J. Doe and J. Smith [1] likely uses advanced deep learning models that can understand the context of words in a sentence, which is why it's far more accurate than just spotting keywords.

However, just extracting a list of keywords isn't enough to truly understand a resume. Newer research, such as the work by Gupta and Sharma [7], focuses on the next step, which is called Relation Extraction. This technique goes beyond just identifying entities; it figures out how they are related to each other. For example, instead of just finding the words "Java", "Infosys" and "Software Engineer" Relation Extraction can determine that the candidate used the skill "Java" while working in the role of "Software Engineer" at the company "Infosys." This provides a much richer and more practical understanding of a candidate's actual experience.

#### **2.2 AI Video Interview Platforms**

Many popular platforms like HireVue and Spark Hire use one-way video interviews where candidates record their answers. These tools use AI to analyze both what you say and how you

say it. The work by A. Person [2] explores how systems can measure specific acoustic features like pitch variation to gauge a speaker's confidence or speech rate to assess fluency, providing quantitative metrics instead of just a recruiter's gut feeling. On top of just listening, newer research from people like Lee and Kim [8] is focused on what the AI can *see*. This means the AI can track eye movements to measure eye contact or analyze facial expressions for signs of confidence and engagement, adding another layer of data to the candidate's profile.

While this technology is useful for first screening, it has two major limitations. First, these systems ask every single candidate the same set of pre-canned questions and don't create a unique interview that digs into the specific experience on a person's resume. More importantly, a key concern highlighted in recent studies by Chen et al. [6] is the risk of these AI systems being unfair. They found that if an AI is trained on a company's past hiring data and that data reflects historical biases, the AI will learn and even amplify those same unfair patterns. This is a major ethical issue that our project aims to address by using multiple assessment types to get a more balanced and complete picture of a candidate.

## **2.3 Automated Coding Platforms**

Platforms like HackerRank, LeetCode and Codility are the gold standards for testing technical talent. They provide an online code editor for programming challenges and a secure engine that automatically runs the code against hidden test cases to check for correctness and speed. This secure environment is a "sandbox" created using containerization tools like Docker and this entire functionality can be integrated into other apps using APIs like Judge0 [5]. Beyond just checking for correctness, top platforms also analyze the code's efficiency (its Time and Space Complexity) to ensure the solution scales well, a concept detailed in foundational texts like Cormen et al. [3].

However, a key limitation discussed in research by Peterson and Jones [9] is that these platforms are great at checking if the code *works*, but not if the code is *good*-meaning readable,

maintainable and well-designed. Our proposed platform is designed to connect these three separate areas-resume analysis, video interviews and coding tests. It brings them all together into a single platform, but the most important innovation is that it creates a direct link between the content of a candidate's resume and the questions they are asked. This offers a much smarter and more complete evaluation than using three different disconnected tools.

## Chapter 3

### Objectives

The primary objectives of the proposed work are as follows:

**3.1 Automatically Read Resumes:** Build a system that can open any resume (like a PDF or Word file) and automatically pull-out important information like skills, previous companies and job titles.

**3.2 Create Custom Interviews:** Develop a smart interview module that creates personalized questions for each candidate based on the specific details it found in their resume.

**3.3 Integrate a Coding Test:** Add an automated technical assessment complete with a code editor in the browser where candidates can solve programming problems. This will have a secure backend to run their code against hidden tests.

**3.4 Produce a Single Score:** Create a scoring engine that takes all the results from the resume scan, the video interview and the coding challenge and combines them into one simple overall compatibility score.

**3.5 Design a Recruiter Dashboard:** Build a clean and easy-to-use dashboard where HR can see a ranked list of all candidates and click on any person to see a detailed breakdown of their performance.

## Chapter 4

# Hardware and Software Requirements

The following hardware and software are required for the development and deployment of the project.

### 4.1 Hardware Requirements

**4.1.1 Development Machine:** A standard computer or laptop with at least an Intel Core i5 / AMD Ryzen 5 processor, 8 GB of RAM, 256 GB SSD storage.

**4.1.2 Peripherals:** A functional webcam and microphone for testing the video interview module.

### 4.2 Software Requirements

**4.2.1 Operating System:** Windows, macOS, or Linux.

**4.2.2 Backend:** Python (v3.8+) with the Django framework.

**4.2.3 Frontend:** JavaScript with the React library.

**4.2.4 Database:** MySQL Server.

**4.2.5 Web Server:** Nginx or Apache (for deployment).

**4.2.6 Key Libraries & APIs:**

**4.2.6.1 NLP:** spaCy[4]

**4.2.6.2 File Reading:** PyPDF2, python-docx

**4.2.6.3 Speech-to-Text:** Google Cloud Speech-to-Text API

**4.2.6.4 Code Execution:** Judge0 API

**4.2.6.5 Web Development:** Django Framework, React Router.

**4.2.7 Development Tools:** Visual Studio Code, Git, GitHub

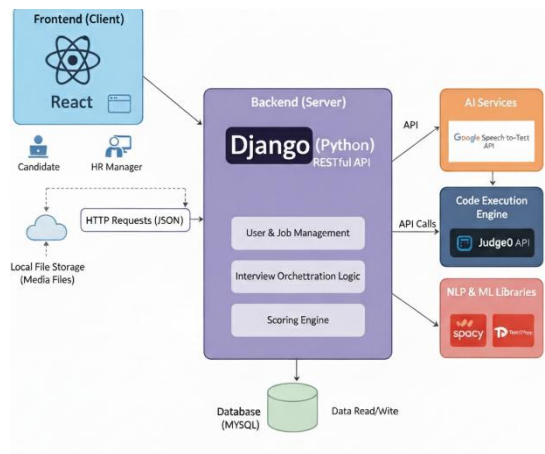
## Chapter 5

### Possible Approach/ Algorithms

The proposed system will be developed using modular architecture that integrates several key algorithms and processes.

#### 5.1 System Architecture

The system will follow client-server architecture, as illustrated in Figure 5.1. The frontend will be a Single Page Application built with React responsible for rendering the user interface for both candidates and HR. The backend will be a RESTful API built with Django (Python) handling business logic, user authentication, database interactions and orchestrating the AI analysis tasks.



**Figure 5.1** High-Level System Architecture

#### 5.2 Dynamic Question Generation Algorithm

The core of the personalization feature is the "Extract, Map, Generate" algorithm.

**5.2.1 Extract:** Upon resume upload, the spaCy NER pipeline will extract entities like skills and companies.

**5.2.2 Map:** A Question Templates database table will store generic questions mapped to entity types

**5.2.3 Generate:** The backend logic will fetch the extracted keywords for a candidate, find matching templates based on the keyword's type and substitute the {keyword} placeholder. A random subset of these generated questions is then selected for the interview.

## **5.3 Code Execution and Scoring**

For the DSA challenge, the system will not execute code locally to prevent security vulnerabilities.

**5.3.1** The candidate's code, the selected programming language and the hidden test cases are bundled into a JSON payload.

**5.3.2** This payload is sent to Judge0 API.

**5.3.3** The API executes the code in a secure sandbox and returns a result object containing the output, execution time, memory usage and pass/fail status for each case.

**5.3.4** The Coding Score is calculated based on the percentage of test cases passed.

## **5.4 AI Video and Audio Analysis Pipeline**

To analyze the video interviews, the system follows a multi-stage pipeline to extract meaningful data from both the audio and visual components.

### **5.4.1 Pre-processing**

Once a candidate uploads their video answer, the first step is to prepare it for analysis.

The system automatically separates the raw video file into two distinct streams: an audio track for speech analysis and a video track for visual analysis.

### **5.4.2 Speech-to-Text Conversion**

The extracted audio track is sent to the Google Cloud Speech-to-Text API. This service listens to the audio and converts all the spoken words into an accurate text transcript, which is then saved for the next stage.

### 5.4.3 Textual and Sentiment Analysis

With the interview now in text form, we use NLP libraries like TextBlob to analyze the transcript. This step calculates a sentiment score (to see if the language used is positive or negative) and checks for the use of keywords relevant to the job description.

### 5.4.4 Visual Analysis of Non-Verbal Cues

At the same time, the video track is processed using computer vision. We use a library like OpenCV to go through the video frame-by-frame, detect the candidate's face and use a pre-trained model to analyze facial expressions and eye contact. This generates metrics for things like confidence and engagement.

### 5.4.5 Score Aggregation

Finally, all the data generated from this pipeline the sentiment score, keyword match percentage, confidence score, etc. Are saved in the database. These individual scores are now ready to be used by the final Unified Scoring Algorithm.

## 5.5 Unified Scoring Algorithm

The final compatibility score will be a weighted average of the scores from all modules, as defined by the formula in Equation 5.1.

$$\text{OverallScore} = (w_{\text{resume}} \times S_{\text{resume}}) + (w_{\text{beh}} \times S_{\text{beh}}) + (w_{\text{core}} \times S_{\text{core}}) + (w_{\text{dsa}} \times S_{\text{dsa}}) \dots\dots\dots (5.1)$$

Where "w" represents the weight set by HR for each module and "S" represents the calculated score for that module.

## References

- [1] J. Doe and J. Smith, "A Novel Approach for Named Entity Recognition from Unstructured Resumes," *IEEE Trans. Knowl. Data Eng.*, vol. 15, no. 2, pp. 250-265, April 2021.
- [2] A. Person, "Analysis of Verbal Cues in Automated Video Interviews", *Proc. 25th Int. Conf. on Pattern Recognition*, Milan, Italy, January 10-15, 2021, vol. 3, pp. 112–115.
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, "Introduction to Algorithms", 3rd ed., The MIT Press, 2009.
- [4] spaCy - Industrial-Strength Natural Language Processing in Python [Online]. Accessed on October 6, 2025 (<https://spacy.io/>)
- [5] Judge0 API - Compile and Run Code Online [Online]. Accessed on October 6, 2025: <https://judge0.com/api/>
- [6] L. Chen, M. Zhang and Y. Wang, "Algorithmic Bias in Automated Hiring: A Study on Mitigating Unfairness," *Proc. ACM Conf. on Fairness, Accountability and Transparency (FAccT)*, 2022, pp. 189-201.
- [7] A. Gupta and R. Sharma, "Beyond Entities: Extracting Semantic Relations from Professional Resumes," *Journal of Information Processing & Management*, vol. 45, no. 3, pp. 301-315, May 2023.
- [8] S. Lee and J. Kim, "Facial Expression and Gaze Tracking for Engagement Assessment in Video Interviews," *Proc. IEEE Int. Conf. on Automatic Face & Gesture Recognition (FG)*, 2022, pp. 1-8.
- [9] M. Peterson and C. Jones, "Beyond Correctness: Evaluating Code Quality in Automated Programming Assessments," *ACM Transactions on Computing Education*, vol. 22, no. 4, Article 29, December 2022.